

Beyond the Syllabus

Sharper Hardness, Solvers That Work Anyway, and Why Lower Bounds Are Hard to Prove
Week 12 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why look beyond the syllabus?

- This week you learned that 3-SAT, VERTEX COVER, CLIQUE, HAMILTONIAN CYCLE, SUBSET SUM, and 3-COLORING are all NP-complete – and a poly. algorithm for one would give one for *all* of them.
- “NP-complete” is not the end of the story – it raises three sharper questions, all active research right now:
 - ① **How exponential, exactly?** $2^{0.001n}$ and 2^n are both “non-polynomial,” but wildly different in practice.
 - ② **If it’s exponential in theory, why do real SAT solvers routinely solve formulas with millions of variables?**
 - ③ **Why is it so hard to even prove these lower bounds?** We have overwhelming *evidence* $P \neq NP$, but no proof – and a precise reason our current techniques cannot supply one.

The throughline

Reductions do more than classify problems as “hard” – they let a hardness *assumption* about one problem (like 3-SAT) transfer precise, quantitative consequences to every problem it reduces to.

Recap: what NP-completeness does *not* tell you

- Cook–Levin plus this week’s reductions prove: *if* $P = NP$, *then* 3-SAT (and Vertex Cover, Clique, ...) has a polynomial algorithm.
- They say nothing about *how exponential* the best algorithm is if $P \neq NP$. Brute-force 3-SAT is $O(2^n)$; is that close to necessary, or could a clever algorithm reach $O(1.01^n)$, or even $O(2^{\sqrt{n}})$?
- This is exactly the kind of question this course trains you to ask about *every* algorithm’s running time – we are just asking it about the *lower* bound side now.

The Exponential Time Hypothesis (Impagliazzo & Paturi)

Impagliazzo & Paturi, “On the Complexity of k -SAT” (CCC 1999; *J. Comput. Syst. Sci.*, 2001)

For $k \geq 3$, let $s_k = \inf\{\delta : k\text{-SAT solvable in } O(2^{\delta n}) \text{ time}\}$. The **Exponential Time Hypothesis (ETH)** conjectures $s_3 > 0$: 3-SAT has *no* $2^{o(n)}$ algorithm.

- A companion **Sparsification Lemma** (Impagliazzo, Paturi & Zane, 2001) shows any k -SAT instance can be re-expressed as $2^{\epsilon n}$ instances with only $O(n)$ clauses each – the technical engine that makes ETH a *robust*, reduction-friendly assumption rather than a fact about one particular encoding.
- ETH is *stronger* than $P \neq NP$ (it rules out sub-exponential time, not just polynomial time) and is now a standard assumption for proving **quantitative** lower bounds via this week’s reduction technique.

Why this matters for problems from this week's lecture

- ETH-based reductions give lower bounds like: VERTEX COVER on an n -vertex graph has no $2^{o(n)}$ algorithm, and INDEPENDENT SET / CLIQUE inherit the same bound through the reductions you proved this week (Lemma: Independent Set $\equiv$ Vertex Cover; Independent Set $\leq_p$ Clique).
- For SUBSET SUM: the pseudo-polynomial $O(nW)$ DP you may see next week is *not* an accident of a weak algorithm – fine-grained results (building on this same ETH/SETH line) show truly-polynomial-in- n -and- $\log W$ algorithms would violate standard hardness assumptions.
- **The upshot:** once you have proved X is NP-complete this week, ETH lets you say something much sharper than “no poly algorithm” – it predicts *how much* exponential blow-up is unavoidable, which is exactly the number you need to decide whether brute force, or the FPT/branch-and-bound tools of Week 13, is the realistic plan.

The puzzle: exponential worst case, routine industrial use

- ETH says 3-SAT has no sub-exponential *worst-case* algorithm. And yet modern SAT solvers are used daily in hardware verification, software model checking, and scheduling, on formulas with **millions** of variables.
- The resolution: worst-case exponential and *practically fast on the instances that actually arise* are compatible. Conflict-Driven Clause Learning (CDCL) solvers combine backtracking search (this week's topic) with learned clauses that prune huge parts of the search space the moment a conflict repeats a pattern.
- Progress on this engineering problem is measured every year at the **SAT Competition** – a genuine, ongoing benchmark of how far practice has outrun the worst-case theory.

SAT Competition 2024–2025: who is winning, and how

Year	Main sequential track winner	Note
2024	Kissat	a from-scratch C port/redesign of CaDiCaL
2025	CaDiCaL-SC2025	PAR-2 2327.00, 161 solved; Kissat-VSA 2nd (2335.54, 160 solved)

- Source: SAT Competition 2025 official results (satcompetition.github.io/2025) and the SAT Competition 2024 medal table (cca.informatik.uni-freiburg.de).
- Striking detail: *all three* medalists in the 2025 main sequential track were Kissat-based variants (augmented with extra heuristics) – the CDCL architecture from this week’s backtracking discussion, refined for 25+ years, is still the state of the art.
- This is the same lesson as the exercises you just did: brute-force backtracking plus the *right* pruning rule is the difference between “theoretically exponential” and “solves the instance in milliseconds.”

We believe $P \neq NP$. Why can't we prove it?

- Every standard technique for proving a Boolean function is *hard to compute with small circuits* has, historically, worked by exhibiting a **property** of hard functions that is: (i) *constructive* (checkable in time polynomial in the function's truth table), (ii) *large* (true for most functions), and (iii) *useful* (false for the easy functions you're separating from).
- **Razborov & Rudich (1994/1997), “Natural Proofs.”** Any proof technique with all three properties would, as a side effect, break standard pseudorandom generators / one-way functions – objects that cryptography needs to exist. So either cryptography is broken, or *no “natural” proof can ever separate P from NP* . This is the **natural proofs barrier**.

2026: the same barrier, for data structures

Loff, Koucký, Molli & Saks, “The Natural Proofs Barrier against Data-Structure Lower-Bounds” (STOC 2026; DOI: 10.1145/3798129.3800843)

The natural-proofs obstruction is not special to circuit complexity: the same style of argument shows that “natural” techniques are also insufficient to prove strong **data-structure** lower bounds (e.g. for dynamic connectivity-style problems closely related to this week’s reductions), under analogous cryptographic-hardness assumptions.

- Why this belongs in a week on NP-completeness: it is a direct descendant of the verification-vs-search asymmetry this week is built on – it says the *proof techniques themselves* face a search-vs-verify-style obstacle, one level up.
- Practical upshot for you: when an exercise asks you to prove a lower bound, “try harder with the natural strategy” sometimes has a hard mathematical ceiling – this is an active 2020s research frontier, not a solved problem.

Summary

- 1 **Sharper hardness.** The Exponential Time Hypothesis (Impagliazzo & Paturi, 1999/2001) upgrades “no polynomial algorithm” to a quantitative $2^{\Omega(n)}$ prediction, transferred to Vertex Cover/Clique/Subset Sum by exactly the reductions you proved this week.
- 2 **Theory vs. practice.** Worst-case exponential and “routinely solves million-variable industrial instances” coexist: SAT Competition 2024–2025 medalists (Kissat, CaDiCaL) are still backtracking search plus learned-clause pruning – this week’s idea, engineered for 25 years.
- 3 **Why lower-bound proofs are themselves hard.** The natural proofs barrier (Razborov–Rudich, 1994/1997) blocks the standard circuit-lower-bound recipe unless cryptography is broken; a STOC 2026 result (Loff, Koucký, Molli, Saks) shows the same obstruction reaches data-structure lower bounds.

Looking ahead

Week 13 (Kleinberg & Tardos, Ch. 10) takes NP-completeness as a **starting point** rather than a dead end: fixed-parameter tractable algorithms confine the exponential blow-up to a small parameter k , PSPACE captures games and quantified logic, and branch-and-bound turns today’s backtracking into a principled search strategy with provable pruning.